

Cricket Shot Quality AI

Pipeline Output & Results — Field-by-Field Reference

What every processing step outputs, and the project's final measured results, in one document.

Vipul Pandey (2023IMG-056) · Avinav Verma (2023IMG-011) · Shubham Raj (2023IMT-076)
Supervisor: Dr. Anshul · ABV-IIIITM Gwalior

September 2026

1. What this document covers

When a clip is uploaded, the app runs it through six processing steps. Each step writes its own output into the clip's data folder, so the full working can be checked at every stage rather than trusting a single final number. This document lists exactly what each step produces — field by field — and shows the real output from an actual clip (cover/video1.mp4) so every value here is real, not an example made up for illustration. Section 8 has the project's final measured results in one table.

This data folder is created for every clip processed, including a user's own uploaded video, and can be downloaded as a zip directly from the app.

2. Pipeline steps, in order

Step	Output file / folder	Produces
1	00_metadata.json	Clip info: resolution, fps, frame counts, detection rate, timings
2	01_extracted_frames/	The 30 frames actually given to the classifier
3	02_skeleton_keypoints.json	All 33 MediaPipe landmarks, per analysed frame
4	03_skeleton_overlay_frames/	Skeleton drawn on the identified striker, per frame
5	04_comparison_frames/	Original frame and skeleton frame side by side
6	05_shot_analysis.json	Prediction, joint angles, quality score, movement curve
—	PIPELINE_SUMMARY.txt	All of the above, in one plain-text summary

Every step runs for every clip — nothing here is a separate script the user has to trigger. The example values in this document all come from one real run: cover/video1.mp4, correctly predicted as flick with 68.2/100 quality.

3. Step 1 — 00_metadata.json

Basic facts about the input clip and how the pipeline handled it.

Field	Type	What it is
video_file	string	Original filename
total_frames	int	Frames in the source video, at its native fps
fps	float	Source video's frame rate
duration_seconds	float	Source video length
resolution	string	Source video resolution, e.g. "1280x720"
extracted_frames	int	Frames actually given to the classifier (fixed at 30)
detection_method	string	Fixed label: "striker-only (YOLO track + crop pose)"
striker_found	bool	Whether a striker track was identified at all
analysed_frames	int	Frames pose estimation ran on
detected_frames	int	Frames where a pose was actually detected
detection_rate	float	detected_frames / analysed_frames, as a percentage
avg_joints_per_frame	float	Average of the 13 cricket joints visible per detected frame
timings_seconds	object	Wall-clock time for each pipeline stage

Real example (this clip):

```
{
  "video_file": "cover_video1.mp4",
  "total_frames": 61,
  "fps": 25.0,
  "duration_seconds": 2.44,
  "resolution": "1280x720",
  "extracted_frames": 30,
  "detection_method": "striker-only (YOLO track + crop pose)",
  "striker_found": true,
  "analysed_frames": 30,
  "detected_frames": 30,
  "detection_rate": 100.0,
  "avg_joints_per_frame": 10.0,
  "timings_seconds": {
    "extract_frames": 0.4,
    "shot_prediction": 8.8,
    "striker_pose": 24.2
  }
}
```

4. Step 2 — 01_extracted_frames/

30 JPEG images, named frame_001.jpg through frame_030.jpg. These are 30 **consecutive** frames read from the start of the clip — the same convention the classifier was trained on. An earlier version of the app extracted 30 strided frames from the clip's middle 60% instead, which fed the model a different kind of input than it was trained on and measurably hurt accuracy (46% top-1 became 32% under that scheme, on the same clips). Each frame is 224×224, padded and resized. ~488 KB total for 30 frames on this clip.

5. Step 3 — 02_skeleton_keypoints.json

One entry per analysed frame (frame_001 ... frame_030). Each entry holds all 33 MediaPipe Pose landmarks for that frame — the 13 cricket-relevant ones are named (nose, left/right shoulder, elbow, wrist, hip, knee, ankle); the other 20 (face detail, fingers, feet) keep MediaPipe's numeric landmark index. Only the 13 named ones are used downstream for angles and quality scoring — the rest are kept in this file for completeness, not discarded before saving.

Field	Type	What it is
x, y	float [0,1]	Position in the frame, normalised to frame width/height
z	float	Relative depth from MediaPipe's 3D world landmarks
visibility	float [0,1]	MediaPipe's own confidence that this landmark is visible

This file has 30 frame entries, 33 landmarks each. One landmark, real values:

```
{
  "frame_001": {
    "left_wrist": {
      "x": 0.5189,
      "y": 0.15609,
      "z": -0.69632,
      "visibility": 0.7516
    }
  }
}
```

6. Steps 4 & 5 — overlay and comparison frames

03_skeleton_overlay_frames/ — the same 30 frames with the skeleton drawn on top: green lines for bone connections, coloured dots for the 13 tracked joints. Only the identified striker gets a skeleton — umpire, non-striker, and any crowd figures are left untouched. Frames with no detected pose are labelled NO_DETECTION rather than showing a guessed skeleton. ~4.9 MB for 30 frames on this clip.

04_comparison_frames/ — original frame and skeleton frame placed side by side, same filename pattern. This is the fastest way to check the skeleton landed on the right player. ~9.5 MB for 30 frames.

Two ready-made videos are also written at the top level of the data folder: skeleton_<name>.mp4 (skeleton only) and comparison_<name>.mp4 (side by side), stitched from these same frames.

7. Step 6 — 05_shot_analysis.json

The final output: what shot was played, how good the technique was, and the full comparison against professional players. This is the largest and most important file in the folder.

7.1 prediction

Field	Type	What it is
shot	string	Predicted shot class (one of the 10 shot types)
confidence	float %	Softmax probability of the top prediction
top3	list	Top 3 predicted shots with their confidences
all_probabilities	object	Confidence for all 10 classes, for full transparency
model	string	Which classifier produced this, with its measured accuracy

```
{
  "shot": "flick",
  "confidence": 39.1,
  "top3": [
    {
      "shot": "flick",
      "confidence": 39.1
    },
    {
      "shot": "cover",
      "confidence": 38.6
    },
    {
      "shot": "lofted",
      "confidence": 18.3
    }
  ],
  "all_probabilities": {
    "cover": 38.56,
    "defense": 0.01,
    "flick": 39.11,
    "hook": 1.18,
    "late_cut": 0.07,
    "lofted": 18.3,
    "pull": 0.42,
    "square_cut": 0.67,
    "straight": 1.64,
    "sweep": 0.04
  },
  "model": "r3d18+effnet (62.4% test top-1, 81.2% top-3)"
}
...
}
```

7.2 joint_angles_at_impact & angles_by_phase

Seven joint angles, measured at the impact frame (peak hand speed) from MediaPipe's metric 3D world landmarks — not the flat 2D image coordinates, which collapse to 155–177° for nearly every shot on this camera angle. `angles_by_phase` repeats the same seven angles at three points: stance, impact, and follow-through, plus which frame number each one came from, the detected handedness (with vote counts), and `clip_averaged_angles_do_not_use` — kept only to show why clip-averaging was abandoned (it produces angles like `back_elbow 2.1°`, a physically impossible reading).

Field	Type	What it is
front_knee_angle	float deg	Knee angle, front leg for the detected handedness
back_knee_angle	float deg	Knee angle, back leg
front_elbow_angle	float deg	Elbow angle, top-hand arm
back_elbow_angle	float deg	Elbow angle, bottom-hand arm
shoulder_tilt_deg	float deg	Shoulder line tilt from horizontal
hip_tilt_deg	float deg	Hip line tilt from horizontal
trunk_lean_deg	float deg	Trunk lean from vertical

```

{
  "handedness": "right",
  "front_knee_angle": 154.8,
  "back_knee_angle": 136.8,
  "front_elbow_angle": 75.6,
  "back_elbow_angle": 99.4,
  "shoulder_tilt_deg": 38.4,
  "hip_tilt_deg": 19.1,
  "trunk_lean_deg": 37.6
}

```

7.3 quality

Score out of 100 against professional ranges for the predicted shot, with a per-criterion breakdown (each criterion's ideal range, this clip's actual value, a 0–100 score, and a status label).

```

{
  "shot_type": "flick",
  "overall_score": 68.2,
  "grade": "Average",
  "criteria": [
    {
      "name": "Front Knee Bend",
      "ideal": "142\u00b0\u2013162\u00b0",
      "actual": "154.8\u00b0",
      "score": 100.0,
      "status": "\u2705 Good"
    },
    {
      "name": "Wrist Flick (Elbow)",
      "ideal": "112\u00b0\u2013144\u00b0",
      "actual": "75.6\u00b0",
      "score": 25.0,
      "status": "\u274c Needs Work"
    },
    {
      "name": "Hip Rotation",
      "ideal": "<15\u00b0",
      "actual": "19.1\u00b0",
      "score": 79.5,
      "status": "\u26a0\u2093 Fair"
    }
  ]
}

```

7.4 vs_professional (impact-instant comparison)

Each of the seven angles at impact, against the median and interquartile range from professional clips of the same shot type. One entry shown (front knee); the real file has all seven.

Field	Type	What it is
label	string	Angle name
actual	float deg	This clip's value
pro_median	float deg	Median across professional clips of this shot
diff	float deg	actual minus pro_median
low, high	float deg	Interquartile range professionals typically fall inside
in_range	bool	Whether actual falls inside [low, high]
n_pro_clips	int	How many professional clips this band was built from

```

{
  "label": "Front knee",
  "actual": 154.8,
  "pro_median": 151.5,
  "diff": 3.3,
  "low": 142.2,
  "high": 161.6,
  "in_range": true,
  "n_pro_clips": 65
}

```

7.5 vs_professional_movement (whole-shot curve)

The newer, whole-shot comparison: each angle resampled onto a fixed 25-point timeline from shot-start to impact, plotted against a professional band built the same way. This is what the app's "Shot movement" section charts. Only the array lengths are shown here (each is 25 numbers, one per timeline point) since the full arrays run long.

Field	Type	What it is
label	string	Angle name
user	float[25]	This clip's angle, resampled shot-start→impact
pro_low, pro_high	float[25]	25th/75th percentile band at each timeline point
pro_median	float[25]	Median band at each timeline point
n_pro_clips	int	Professional clips this band was built from
start_frame, impact_frame	int	Which frames of this clip bound the window

```

"front_knee_angle": {
  "label": "Front knee",
  "user": [150.9, 150.69, ... 25 values, ending 154.8],
  "pro_low": [138.2, ... 25 values],
  "pro_median": [148.3, ... 25 values],
  "pro_high": [159.0, ... 25 values],
  "n_pro_clips": 55,
  "start_frame": 13, "impact_frame": 16
}

```

8. Final measured results (project-wide)

These are not per-clip numbers like the sections above — this table is the project's overall, final, measured performance, from the held-out test split and the verification passes done separately.

Metric	Result
Shot classification, top-1	62.4% (250-clip held-out test split)
Shot classification, top-3	81.2%
Original inherited claim	94% — re-measured, actual was 57.6%
Striker identification	0 wrong-player frames out of 416 checked, all 10 shot classes
Pose detection	100% on 9 of 10 reference clips
Quality score, pro sweep clip	10.2 → 76.0 / 100, after fixing the scoring pipeline
Inference speed	~25–33s per 8–10-second clip, CPU only, no GPU

Seven techniques tried to push accuracy past 62.4%

All measured, all flat or negative — recorded so they are not re-attempted from scratch without new data.

Technique	Result vs. baseline
Pooled skeleton fusion (4 variants)	-4.0 to +3.6 pts
ST-GCN skeleton-graph fusion	-0.8 pts
Class-weighted retraining	+0.0 pts
Ensembling (2 seeds)	+0.0 pts
Multi-window inference	-3.6 pts
Partial EfficientNetB0 fine-tune	-7.2 pts (overfit)

Conclusion: the 1250-clip training set is the binding constraint on accuracy, not the model architecture. Full detail, methodology, and honest failure analysis for each technique is in the project README.

Every value in this document is pulled directly from real files: a genuine pipeline run of cover/video1.mp4 through the live app (Sections 3–7), and the project's own measured test-set results and experiment logs (Section 8). Nothing has been estimated or invented for this report.